

Training Guide

Build, run the deterministic workflow, train, and produce a deployable invalid-traffic network

A complete, copy-paste walkthrough of the training half of the pipeline: install the toolchain, build the platform, run the deterministic ad-fraud workflow, bring your own data, and produce the ready-to-deploy multi-label network the Deployment Guide consumes.

Document	Model Training Guide
Product	Jneopallium Ad-Fraud Guardian (advertising-fraud module)
Model	advertising-fraud 1.0.0 · macro-F1 $\approx$ 0.97
Trainer	scripts/demo-ad-fraud/run_all.py
Output	Deployable JNeopallium layer/neuron config
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Where training ends and deployment begins
3. Prerequisites and system requirements
4. Step 1 — Get the source and build
5. Step 2 — Run the deterministic workflow
6. Step 3 — Understand the generated outputs
7. Step 4 — The workflow stages, explained
8. Step 5 — Bring your own data (the canonical event)
9. Step 6 — The generated artifacts: a deployable network
10. Step 7 — Verify the run against the gates
11. Troubleshooting
12. Appendix — training cheat-sheet

1 What you will achieve

By the end of this guide you will have: a built Jneopallium platform; a fully reproducible ad-fraud workflow run; a freshly trained, calibrated **multi-label invalid-traffic model**; and — the key output — a **complete, deployable JNeopallium network** (eight layer files, real runtime classes, embedded calibrated heads, and an advisory response gate) that the companion **Deployment Guide** loads directly.

Safety ceiling for the whole pipeline

Everything the trained model does is advisory. It produces candidate actions and evidence; it never blocks a publisher, withholds a payout, or accuses a person. The readiness report keeps `AUTOMATED_ACTION_READY` false until first-party labelled production traffic, legal review, and an appeal/rollback process are recorded.

2 Where training ends and deployment begins

Phase	What happens	Document
Training (this guide)	Build, run the workflow, train and calibrate, emit the network	Training Guide
Deployment	Load the emitted network, configure event sources, run the advisory scorer	Deployment Guide

The hand-off is a single directory of generated artifacts (Step 6). Training **produces** it; deployment **consumes** it. The trainer writes JNeopallium-ready layer and neuron configuration with the trained heads, calibration, and thresholds inside, so the files that record your run are the files the runtime boots from.

3 Prerequisites and system requirements

Toolchain

Tool	Version	Why
Java JDK	17 or newer (LTS)	Build and run the worker / runtime scorer
Apache Maven	3.9 or newer	Multi-module build of master + worker
Python	3.10+ (3.13 tested)	Runs the workflow; no third-party packages required
Git	any recent	Clone the repository

The workflow runs fully **offline and deterministically** — it does not ask for manual files. Optional public sources that are unavailable, license-blocked, or credential-gated are simply recorded as such; the deterministic simulator supplies coverage for the fraud classes that lack complete public labels.

Verify the toolchain

```
java -version      # expect 17 or newer
mvn -version       # expect 3.9 or newer
python --version   # expect 3.10 or newer
```

4 Step 1 — Get the source and build

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

The build installs `com.rakovpublic.jneopallium:worker:1.0`. The worker already contains the real ad-fraud neuron, processor, and signal classes the trained network references (`com.rakovpublic.jneopallium.worker.net.neuron.impl.adfraud.*` and friends), so no extra hand-written classes are required to train.

5 Step 2 — Run the deterministic workflow

One command performs the entire workflow: source discovery, deterministic scenario generation, normalization, leakage audit, training, calibration, evaluation, model export, Java tests, demo replay, and a readiness report.

Linux / macOS

```
scripts/demo-ad-fraud/run_all.sh
```

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-ad-fraud/run_all.ps1
```

Common flags

Flag	Effect
<code>--offline</code>	Never attempt network downloads; record sources as unavailable
<code>--quick</code>	Smaller, faster corpus (still ≥ 60 events per scenario)
<code>--max-rows N</code>	Corpus size (default $1900 \approx 100$ events per scenario)
<code>--seed N</code>	Deterministic seed (default 1729)
<code>--skip-java</code>	Skip the Maven Java tests (useful while iterating)

Example: `python scripts/demo-ad-fraud/run_all.py --offline`. The run prints a JSON summary with the macro-F1, the readiness flags, and the Java-test status.

6 Step 3 — Understand the generated outputs

Workflow evidence is written under `target/jneopallium-ad-fraud/`, and the deployable model bundle is written to `worker/src/main/resources/model/advertising-fraud/`. The summary looks like:

```
{
  "macroF1": 0.966132,
  "readiness": {
    "ENGINEERING_READY": true, "SHADOW_READY": true,
    "ADVISORY_READY": true, "AUTOMATED_ACTION_READY": false
  },
  "javaTests": "passed"
}
```

Readiness flag	Meaning
ENGINEERING_READY	The pipeline is wired, reproducible, and exports a valid bundle
SHADOW_READY	Safe to run read-only beside production, writing advisories to an audit sink
ADVISORY_READY	Safe to surface advisories to analysts
AUTOMATED_ACTION_READY	Always false until first-party labels + legal review + appeal/rollback

7 Step 4 — The workflow stages, explained

- 1. Source discovery & bounded download.** Public sources (e.g. Criteo attribution / click logs, ads.txt / sellers.json crawl, benign crawler identifiers) are catalogued; unavailable or credential-gated ones are recorded, not requested.
- 2. Deterministic scenario generation.** A seeded simulator produces realistic events across ~19 scenarios (bots, click farms, incentivized cohorts, click spam/injection, postback forgery, inventory spoofing, plus legitimate and accidental traffic).
- 3. Normalization & leakage audit.** Events become canonical rows; the audit fails the run if any device / click / campaign identifier crosses split boundaries.
- 4. Training & calibration.** Class-weighted, L2-regularized logistic heads are fitted over base + behavioural-evidence + non-linear interaction features, then each head is Platt-calibrated on a held-out calibration split.
- 5. Cost-aware thresholds & evaluation.** Per-label operating points are chosen by expected utility; metrics include precision/recall/F1, PR/ROC-AUC, Brier, calibration error, and cost-weighted savings.
- 6. Export, tests, readiness, docs.** The deployable network is written, Java tests run, and a readiness report and model/data cards are produced.

8 Step 5 — Bring your own data (the canonical event)

To move beyond the reference corpus you supply your own canonical events. Each event keeps both event and ingest time and carries integrity, behavioural, supply-chain, and delayed-quality fields, plus label-

provenance metadata. Raw personal identifiers must be HMAC-hashed before training, persistence, or logs.

The split policy is the integrity guarantee

Never split by random row. Split by scenario, campaign episode, and deterministic replicate block, and keep an adversarial holdout — so reported quality reflects genuinely unseen traffic. The leakage audit enforces this automatically.

Label provenance matters: the reference model uses deterministic simulator labels and weak public-source metadata. Real production accuracy requires **first-party labelled traffic** — confirmed chargebacks, MMP fraud rulings, manual analyst verdicts — and forward-time and unseen-publisher validation.

9 Step 6 — The generated artifacts: a deployable network

Every run writes a directory of JNeopallium-style configuration that is ready to deploy as-is — the actual layer-and-neuron network the runtime boots from:

Artifact	What it is
model-descriptor.json	Whole-network descriptor: 8 layers, 22 neurons, frequency map, runtime classes
fallback-model.json	The trained calibrated heads + the non-linear hidden layer (featureNames, hidden, heads)
layer-0.json ... layer-6-..., result-layer.json	The eight deployable layer files
thresholds.json / calibration.json	Cost-aware per-label thresholds and Platt calibration
feature-schema.json / label-schema.json	The input features and the ten labels
training-manifest.json	Sources, example count, split policy, full metrics
MODEL_CARD.md / DATA_CARD.md	Safety cards (advisory-only; source statuses)
checksums.sha256	Integrity manifest the Java bundle verifies on load

model-descriptor.json records the **8-layer, 22-neuron** network — input, event-integrity, entity/graph/quality, behavioural-evidence, feature-interaction (the non-linear hidden layer), trained correlation, advisory response gate, and result — every neuron referencing a concrete class from the worker's ad-fraud package.

Why this matters

Because training emits real, inspectable, deployable configuration with the calibrated heads and thresholds inside, there is no translation step between "the model we trained" and "the network we run." The Deployment Guide simply points the runtime at this directory.

10 Step 7 — Verify the run against the gates

1. The leakage audit passed (no identifier crosses split boundaries).
2. Macro-F1 meets your bar (the reference run reaches ≈ 0.97 ; the per-label table is in the descriptor and the Test Report).

3. Calibration is real Platt scaling per label (not identity), and calibration error is within budget.
4. The eight layer files, the descriptor, and checksums.sha256 were written and reference the expected runtime classes.
5. The Java tests pass (javaTests: passed) and the readiness report keeps AUTOMATED_ACTION_READY=false.

Reproducibility check

The run is deterministic for a given seed. Re-running the same command must reproduce the same macro-F1 and the same per-label metrics. A drift signals that an input, the seed, or the toolchain changed.

11 Troubleshooting

Symptom	Likely cause and fix
java -version shows < 17	Install a JDK 17+ and set JAVA_HOME
Workflow exits at the leakage audit	An identifier crosses splits; check the split policy in your data
Java tests fail after a change	Structural counts changed; re-export the bundle and align the test expectations
Sources show dataset_unavailable	Expected offline; the deterministic simulator still provides coverage
Macro-F1 lower than expected on your data	Add first-party labels and more per-scenario variety; check calibration
Bundle checksum mismatch in Java	Re-run the workflow so checksums.sha256 matches the exported files

12 Appendix — training cheat-sheet

Build

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean install
```

Run the workflow

```
# full deterministic run (offline)
python scripts/demo-ad-fraud/run_all.py --offline

# quick iteration, skip Java tests
python scripts/demo-ad-fraud/run_all.py --quick --offline --skip-java

# Linux/macOS and Windows wrappers
scripts/demo-ad-fraud/run_all.sh
powershell -File scripts/demo-ad-fraud/run_all.ps1
```

Jneopallium Ad-Fraud Guardian · Training Guide. The trainer emits a deployable JNeopallium network; see the Deployment Guide to run it. Safety mode: ADVISORY / SHADOW. License: BSD 3-Clause.